

Desarrollando de habilidades blandas en etapas tempranas en la formación de Ingenieros de Software

Elizabeth Vidal¹, Ricardo Gacitúa², Mauricio Dieguez²

evidad@unsa.edu.pe, ricardo.gacitua@ufrontera.cl, mauricio.dieguez@ufrontera.cl

¹ Universidad Nacional de San Agustín de Arequipa, Arequipa, Perú.

² Universidad La Frontera, Temuco, Chile.

Pages: 423–436

Resumen: Los requisitos de acreditadoras internacionales en carreras de Ingeniería como ABET además del dominio de los conocimientos técnicos hacen hincapié en la importancia de las habilidades “blandas”. En la industria del software el desarrollo de estas habilidades se hace mucho mas necesario debido a que la propia naturaleza del proceso de desarrollo de software es intensivo en comunicación y trabajo en equipo. La evidencia muestra que los estudiantes de los primeros años de carreras de computación son enseñados sin considerar en forma explícita el desarrollo de las habilidades blandas. Una estrategia para enfrentar este desafío, es incluir el uso de metodologías sistemáticas de enseñanza de programación, que no solo se centren en aspectos del proceso, sino que también consideren explícitamente las habilidades blandas. Este artículo describe como la metodología R&G, considera un conjunto de habilidades blandas en un primer curso de programación. Los resultados iniciales demuestran que es posible considerar metodologías que, tempranamente, aborden las habilidades blandas requeridas por los estudiantes relacionadas a carreras de desarrollo de software.

Palabras-clave: programación, enseñanza, habilidades blandas, R&G Methodology

Developing soft skills in early stages studies of Software Engineers

Abstract: The requirements of international accreditors in Engineering careers such as ABET in addition to mastery technical knowledge emphasize the importance of “soft” skills. In the software industry, the development of these skills becomes much more necessary because of the nature of the software development process is intensive in communication and teamwork. Evidence shows that students in the first years of computer careers are taught without explicitly considering the development of soft skills. A strategy to face this challenge is to include the use of systematic programming teaching methodologies, which not only focus on aspects of the process but also explicitly consider soft skills. This article describes how the R&G methodology considers a set of soft skills in a first programming course. The

results show that it is possible to consider methodologies that, early on, address the soft skills required by computer students.

Keywords: programming, teaching, soft skills, R&G Methodology.

1. Introducción

En el contexto de desarrollo de software existe un creciente interés por el desarrollo de las denominadas habilidades blandas (“soft skills”, en inglés) producto de que el desarrollo de software enfrenta un escenario cambiante y cada día más complejo (Ahmed, Capretz, Bouktif, & Campbell, 2013; Hazzan & Har-Shai, 2013). Estas denominadas habilidades blandas se refieren a habilidades intangibles y no-técnicas que se requieren de los profesionales, por tanto son habilidades menos especializadas, menos arraigadas en vocaciones específicas y más alineadas con la disposición general y la personalidad de una persona. Ejemplos de estas habilidades son comunicación, trabajo en equipo y resolución de problemas. La necesidad de desarrollar nuevas competencias en los estudiantes, por lo tanto, es un factor clave para enfrentar el escenario tecnológico actual (Denning & Gordon, 2015).

Con una alta demanda por desarrollar habilidades relacionadas con programación, hay un alto interés por aprender a codificar (Chilana et al., 2015). Sitios tales como code.org, codecademy.org y khanacademy.org, reciben a miles de personas enrolándose en diferentes alternativas de aprendizaje. Las nuevas tecnologías y especialmente las necesidades dinámicas de la demanda laboral, han permitido el surgimiento de un número creciente de nuevos enfoques de aprendizaje, los cuales tienen como objetivo que los estudiantes se entrenen de forma eficaz y a mayor velocidad en áreas puntuales de conocimiento. Cada vez es más frecuente que dichos enfoques tomen la forma de MOOCs, es decir cursos abiertos masivos y dispuestos en internet; los “Nanotítulos”, que son cursos intensivos cortos, altamente prácticos que ofrecen certificación reconocida por compañías de distintos rubros (e.g. Amazon AWS, Google, etc.); las academias que usan técnicas de blended learning; y finalmente los Bootcamps, que son actividades inspiradas en los campos de entrenamiento que las fuerzas armadas utilizan para preparar sus tropas (Burke, Bailey, Lyon, & Green, 2018).

Lamentablemente, la enseñanza de programación continúa siendo un desafío para los educadores, la evidencia empírica demuestra que un importante número de estudiantes de computación son enseñados de forma indisciplinada, la que está enfocada principalmente en aspectos del producto (e.g. código y estructuras), por sobre aspectos del proceso (e.g. conceptualización y modelamiento) (Gacitúa, Diéguez, & Vidal, 2017; Vidal, Gacitúa, Diéguez, & Chachero, 2019). En efecto, Parnas (Parnas, 2010) advierte acerca de los peligros del desarrollo de software indisciplinado y establece que los estudiantes de computación no son enseñados para trabajar en forma disciplinada. Es frecuente escuchar hablar acerca de la “enseñanza creativa” o que el desarrollo de software es un “arte”, esto quizás como una forma de explicar la falta de procedimientos sistemáticos en la construcción de productos de software en etapas tempranas. Muchos estudiantes aprenden de lenguajes de programación y estructuras de código, pero tienen enormes dificultades para analizar problemas y posteriormente convertirlos en una solución escrita en un lenguaje de programación, para trabajar en conjunto con

otras personas y/o explicar y comunicar su solución. Por tanto, en los enfoques antes mencionados, el desarrollo de habilidades blandas no forma parte de su estrategia de enseñanza (Makasiranondh, Maj, & D, 2011; Tomić et al., 2019). Milne and Rowe (Milne & Rowe, 2002) destacan que una de las razones que hacen difícil la programación es que los estudiantes carecen de un modelo claro que les permita comprender el flujo de tareas para crear un programa de computador. Por su parte, Kirschner et al. (Kirschner, Sweller, & Clark, 2006), sugiere que, basado en el conocimiento actual de la arquitectura cognitiva humana, la instrucción que provee mínimas guías, pareciera ser no efectiva. En efecto, se asegura que el aprendizaje requiere de representaciones mentales y esquemas los cuales están sujetos a cuánta información es provista. Es decir, a mayor información provista mejor será la calidad de las representaciones mentales. No existe evidencia, por tanto, que sugiera que el proveer poca información permita realizar mejores construcciones mentales. Hazzan et al. (Hazzan & Har-Shai, 2013) establece que las habilidades blandas (por ejemplo, trabajo en equipo, resolución de problemas complejos y comunicación), no pueden ser estudiadas y enseñadas formalmente, más bien, deben aprenderse y comprenderse gradualmente a lo largo de un cierto período de tiempo, sobre la base del compromiso de los estudiantes, el aprendizaje activo y la reflexión. Este es un factor clave, considerando que muchos de los enfoques antes mencionados, solo favorecen el trabajo individual, aislado y centrado principalmente en aprendizaje de un lenguaje de programación. Por tal razón, es evidente que dichos enfoques, no proveen evidencia empírica de cómo las habilidades blandas requeridas por la industria están siendo consideradas.

De modo de enfrentar este problema, parece razonable pensar que utilizando apropiadas metodologías sistemáticas de enseñanza de programación, que consideren explícitamente las habilidades blandas en etapas tempranas, tales como trabajo en equipo, resolución de problemas y comunicación, se provee de mayor y mejor información que posibilite la construcciones de representaciones mentales y esquemas que permitan centrarse en analizar primeramente los problemas, y posteriormente, mediante un proceso guiado que considera el desarrollo de soluciones en equipo, se pueda obtener la solución representada en líneas de código y por consiguiente, fortalecer el desarrollo de las habilidades blandas requeridas para el desarrollo de software. Adicional a lo anterior, se debe considerar que en el creciente escenario de internacionalización y acreditación de las universidades, y por consiguiente la acreditación de sus programas de estudios, comienza a adquirir relevancia el proveer alguna evidencia del logro de habilidades blandas en los estudiantes. En particular, en el caso de los programas relacionados con computación, instituciones de acreditación, tales como ABET¹, buscan evidencia del logro de objetivos educacionales y habilidades blandas, por sobre la evidencia relacionada con la existencia de documentos o procedimientos.

Este artículo describe la forma en que una metodología utilizada para enseñar a programar, denominada metodología R&G (Rápida & Global), considera y fortalece el desarrollo de un conjunto de habilidades blandas, tales como resolución de problemas, trabajo en equipo y comunicación, en un primer curso de programación, es decir en etapas tempranas. Incluir en forma explícita el desarrollo de habilidades blandas en los primeros cursos de programación se constituye en un primer paso para hacer del

¹ <https://www.abet.org/accreditation/>

proceso de enseñanza de programación un proceso repetible y sistemático que permita fortalecer la construcción de representaciones y esquemas mentales y el desarrollo en forma temprana de habilidades requeridas por la industria de desarrollo de software.

La estructura del artículo es como sigue: La sección 2 presenta la terminología asociada a las habilidades blandas requeridas en las carreras de ciencias de computación. Se describe además, la propuesta metodológica R&G, así como sus principales características y habilidades blandas explícitamente definidas. La sección 3 presenta un marco de análisis para analizar cómo una metodología de programación aborda el desarrollo de las habilidades blandas requeridas por la industria. Luego, la sección 4 presenta los resultados y una explicación de cómo la metodología R&G fortalece el desarrollo de las habilidades blandas. La sección 5 presenta una breve discusión respecto de los resultados obtenidos. La sección 6 destaca algunas limitaciones de presenta este estudio. Finalmente la sección 7 resume las principales conclusiones y trabajos futuros.

2. Background

La presente sección presenta la terminología asociada en esta propuesta, en especial definiciones de habilidades blandas. Se presenta además, la propuesta metodológica R&G, así como sus principales características y habilidades explícitamente definidas.

2.1. Requerimientos actuales de la Industria

Las exigencias relacionadas al desarrollo de software han cambiado ampliamente, no solo porque la complejidad de los sistemas se ha incrementado, sino porque las aplicaciones de software han variado. El surgimiento de nuevas tecnologías de software, tales como cloud computing; analítica y visualización; integración de dispositivos; equipos móviles; aprendizaje de máquinas; procesamiento de lenguaje natural; obliga a incluir nuevas características al proceso de desarrollo y a los productos, tales como escalabilidad, resistencia frente a fallos, auto-adaptación, entre otros aspectos (Hohpe, Ozkaya, Zdun, & Zimmermann, 2016). Estos tipos de sistemas requieren la participación de equipos de desarrollo conformados por líderes de proyectos, arquitectos, analistas, programadores e ingenieros de pruebas, entre otros. Incluso, muchas veces estos equipos se encuentran distribuidos en diferentes partes del mundo. Empresas tales como Google, Facebook, Spotify, Microsoft, Belatrix, Amazon, Apple, destacan entre sus perfiles habilidades tales como la habilidad de trabajo en equipo y comunicación.

Un importante estudio presenta la relación de las habilidades blandas y el dominio de ingeniería de software y sistemas de información basado en anuncios de trabajo publicados en sitios de reclutamiento conocidos ubicaciones geográficas que incluyen América del Norte, Europa, Asia y Australia. El estudio identifica nueve habilidades blandas dentro de la industria del software (Robles, 2012).

2.2. Habilidades blandas

Según Kamin (Kamin, 2013), las habilidades blandas son habilidades interpersonales que demuestran la capacidad de una persona para comunicarse de manera efectiva y construir relaciones con otros en interacciones uno a uno o en grupos y equipos. Para este autor, la práctica de estas habilidades ayuda en la comunicación y promueven la

resolución de problemas, negociación, resolución de conflictos y trabajo en equipo. De los cinco criterios que establece ABET, a través de la Computing Accreditation Commission (CAC), en su nueva versión (2019-2020), dos de ellos se refieren a (1) la habilidad de comunicarse efectivamente en una variedad de contextos profesionales y (2) la habilidad de funcionar efectivamente como miembro o líder de un equipo involucrado en actividades apropiadas a la disciplina del programa.

2.2.1. Habilidad de Comunicación

La habilidad de comunicación se refiere a la habilidad de transmitir ideas con facilidad y claridad de manera que el equipo de trabajo pueda avanzar hacia un objetivo común. Los desarrolladores de software deben tener la habilidad de transmitir ideas complejas en forma sencilla, articular con claridad cuál es el problema a resolver y el objetivo a lograr (Belzer, 2004). Dentro del ámbito de las carreras de computación, La ACM/IEEE-CS2013 expresa que se espera que los graduados deben tener la capacidad de hacer presentaciones efectivas a una variedad de audiencias sobre problemas técnicos y sus soluciones (“ACM Curricula Recommendations: Computer Science Curricula 2013,” n.d.).

2.2.2. Habilidad de Trabajo en Equipo

El trabajo en equipo involucra un grupo de personas que trabajan juntas de manera formal para garantizar el logro de objetivos definidos. En lo referido al desarrollo de software existe una considerable interacción entre las personas que llevan a cabo diversos roles y funciones pero con un propósito común. De acuerdo a la ACM/IEEE-SwEngineering2014, los graduados en ingeniería del software deberán ser capaces de trabajar tanto individualmente y como parte de un equipo para desarrollar y entregar artefactos de software de calidad. Para ello, durante sus estudios deben realizar tareas que impliquen experimentar diversos roles tan explícitamente como sea posible. Esto debe incluir un énfasis en la importancia de asuntos tales como un enfoque disciplinado, cumplir con los plazos, la comunicación y evaluaciones de desempeño individual y de equipo (Ardis et al., 2015).

2.2.3. Habilidad de Resolución de Problemas

La resolución de problemas es un proceso cognitivo dirigido al logro de un objetivo cuando no existe un método de solución conocido ni evidente para quien resuelve el problema (Lovett, 2002; Mayer & Wittrock, 2006). La ACM/IEEE-CS2013 define la habilidad de resolución de problemas como el poder aplicar los conocimientos que han adquirido los estudiantes para resolver problemas reales y no solo escribir código y mover bits. Además, deben darse cuenta de que hay múltiples soluciones a un problema dado y que seleccionar entre ellos no es una actividad puramente técnica, ya que estas soluciones tendrán un impacto real en la vida de las personas (“ACM Curricula Recommendations: Computer Science Curricula 2013,” n.d.).

2.3. R&G Methodology

La metodología R&G (Rápida y Global) (Gacitúa et al., 2017), es una propuesta metodológica para la enseñanza de la programación que impulsa el desarrollo de soluciones a través de un proceso sistemático, disciplinado y repetible. Esta metodología

busca el desarrollo de habilidades cognitivas para la resolución de problemas, centrándose en la enseñanza de un proceso que guíe la resolución de un problema. No busca que un estudiante logre un alto dominio de la sintaxis de algún lenguaje de programación, sino que el estudiante sea capaz de proponer soluciones a un problema, de manera sistemática y disciplinada. La metodología consiste de 4 etapas:

- *Define*: En esta etapa se realiza un análisis del problema, con el objetivo de asegurarse que el estudiante logre una comprensión del problema al que se enfrenta y sea capaz de establecer las principales estructuras y componentes de la solución a proponer.
- *Resuelve*: En esta etapa se plantea el diseño de la solución, a través de un lenguaje gráfico, en el cual se detallan las actividades que dan solución al problema. En esta etapa, se logra obtener el conjunto de instrucciones que dan solución al problema.
- *Convierte*: En esta etapa se convierte el diseño propuesto en la etapa anterior, a través de un conjunto de reglas de transformación, en un algoritmo escrito en pseudo-código. Para lograr esto, se recorre el árbol generado en la etapa de diseño transformándolo en su equivalente en pseudo-código, a través de un conjunto de reglas de transformación.
- *Codifica*: En esta última fase, se construye el código de la solución, en cualquier lenguaje de programación, a partir del pseudo-código obtenido en la etapa anterior. Al igual que en la fase anterior, se puede automatizar la obtención del código.

La metodología busca desarrollar en los estudiantes un conjunto de competencias o habilidades, centrando su foco en aquellas asociadas al análisis y diseño de soluciones. Sin embargo, también aporta al desarrollo de habilidades blandas como el trabajo en equipo, comunicación, resolución de problemas, entre otras. En la tabla-1, resumimos estas habilidades blandas, de acuerdo a la clasificación que se presenta en (Ahmed et al., 2013), indicando para cada una, la fase de la metodología donde se trabajan.

	Etapas de la Metodología			
	Etapas 1	Etapas 2	Etapas 3	Etapas 4
Adaptación a los cambios		✓	✓	✓
Trabajo independiente			✓	✓
Trabajo en equipo	✓	✓		
Habilidades organizacionales	✓	✓		
Habilidades para resolver problemas	✓	✓		
Habilidades interpersonales		✓		
Habilidades de Comunicación	✓	✓		

Tabla 1 – Habilidades blandas según etapas de la metodología R&G.

Durante los dos últimos semestres se ha utilizado la metodología en algunos módulos de una asignatura de programación, obteniéndose impactos positivos en (Diéguez, Gacitúa,

& Vidal, 2018): i) los promedios de notas de los estudiantes, ii) las tasas de aprobación, iii) la homogeneidad de las notas (los rangos de notas es mucho más homogéneo), iv) las tasas de deserción y v) la adopción, por parte de los estudiantes, de un proceso sistemático y disciplinado para plantear soluciones a problemas de programación y vi) el desarrollo de un conjunto de habilidades blandas.

3. Marco de Análisis

Esta sección presenta un marco de análisis el cual permite proveer de los aspectos relevantes para discutir la manera en que se aborda el logro de ciertas habilidades blandas en la metodología R&G. Dicho marco consta de 3 categorías, tal como lo ilustra la figura-1, las cuales se analizarán en el contexto de la metodología. Estas categorías son:

- *Habilidad*. Se refiere a las habilidades intangibles y no-técnicas que se requiere desarrollar (habilidades blandas). En este caso, los estudiantes conocen de antemano la habilidad sobre el cual están trabajando. Por ejemplo, trabajo en equipo y comunicación.
- *Práctica*. Se refiere al conjunto de actividades que permiten aprender y comprender gradualmente la habilidad a lo largo de un cierto período de tiempo, sobre la base del compromiso de los estudiantes, y el aprendizaje activo.
- *Perspectiva*. Las perspectivas que los estudiantes tienen sobre el mundo que les rodea y sobre sí mismos cuando comprenden estas habilidades y se involucran en estas prácticas. La perspectiva comprende la reflexión sobre esta habilidad y entender porqué se logra o no se logra la habilidad y cómo su adquisición puede ser mejorada.

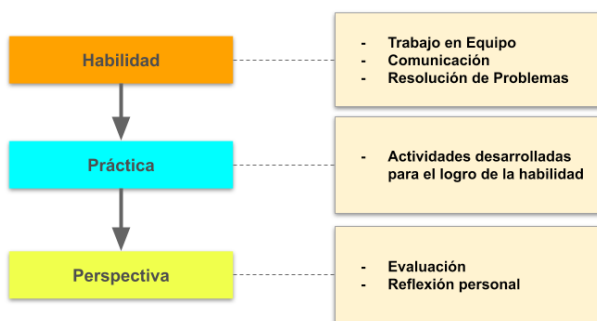


Figura 1 – Marco de análisis.

El análisis de cada habilidad blanda considera los siguientes elementos:

- *Trabajo en Equipo*. Esta habilidad se refiere al trabajo realizado por varias personas, en donde cada uno hace una parte pero todos tienen un objetivo común. Algunos aspectos a considerar en el análisis de esta competencia son: i) existencia de comunicación abierta a objeto de evitar los conflictos, ii) Nivel de

Coordinación para evitar confusiones, iii) Nivel de Cooperación en el desarrollo de las tareas, y iv) Nivel de interdependencia para mantener los niveles de confianza y rendimiento.

- *Comunicación.* La comunicación, como habilidad blanda, es la acción consciente de intercambiar información entre dos o más participantes con el fin de transmitir o recibir información u opiniones distintas. En este caso, se considera los actos de escribir, escuchar y hablar.
- *Resolución de problemas.* Consiste en analizar problema informático complejo y aplicar los principios de la informática y otras disciplinas pertinentes para identificar soluciones. En este caso, se considera la identificación del problema y su modelado como elementos relevante de la resolución de problemas.

4. Resultados

Esta sección analiza los aspectos relevantes de la metodología R&G y explica cómo se aborda el logro de cada habilidad blanda definida en el marco de análisis.

4.1. Aspectos Preliminares

La experiencia descrita en esta sección, corresponde a un primer curso de programación en la Universidad Nacional de San Agustín de Arequipa, Perú, conformado por 30 estudiantes, en el año 2019. Dicho curso tiene una duración de 17 semanas. El desarrollo de los primeros ejercicio práctico corresponde a ejercicios de carácter individual. Dichos ejercicios son simples y abordan procesamientos aritméticos, condicionales simples, y bucles (4 semanas). A partir de la quinta semana se presentan ejercicios de mayor complejidad. A partir de la quinta semana se les solicita a los estudiantes que hagan uso de la Metodología R&G en su primera y segunda etapa (Define y Resuelve).

4.2. Análisis

En esta sección se detalla la forma en que la metodología R&G aborda el desarrollo de las habilidades blandas mencionadas anteriormente, y se describen las estrategias que se utilizaron para su desarrollo, las etapas en las que se trabajaron y los resultados que se obtuvieron.

Trabajo en Equipo:

Práctica. Para enfrentar el desarrollo de esta habilidad, los estudiantes trabajaron en grupo cada problema propuesto, debiendo auto-organizarse a través de la asignación de roles dentro del grupo, donde uno actúa como el jefe de grupo, asignando y dirigiendo las tareas del resto. El jefe del grupo es el interlocutor del grupo frente al profesor, además es el responsable de la coordinación e integración del trabajo de cada integrante y del resultado obtenido. El resto de los integrantes, son los responsables de las tareas individuales, preocupándose de la resolución de su problema particular así como de la posterior integración de su solución. Esto implica que, aunque el jefe del grupo sea el responsable final y quien tiene la misión de integrar, cada integrante debe ser partícipe del diseño general. Este esquema se replica en cada ejercicio, variando los roles de los

integrantes. En la figura-2, se ilustra el trabajo de los grupos en la resolución de un problema dado.

Perspectiva. Para evidenciar resultados se realizó una encuesta en la que se pregunta a los estudiantes que forma de trabajo creen que les permite llegar de mejor manera a la resolución del problema, si de la forma de trabajo individual o el desarrollo en grupal. El total de estudiantes contestó que haciendo trabajo en equipo, como se muestra en la figura-3. Algunas de las respuestas que se obtuvieron fueron: “en grupo, porque me permite ver otras perspectivas”, “en equipo porque me ayuda a darme cuenta de los temas que me faltan reforzar”, “en grupo, porque la discusión nos ha permitido comprender mejor el problema, cada uno aporta y explica como lo comprendió” o “en equipo porque aprendo de los que más saben”.



Figura 2 – Trabajo en equipo

Para resolver un problema prefiero:



Figura 3 – Resultados.

Comunicación:

Práctica. El desarrollo de esta habilidad se consideró desde tres dimensiones: Escribir, escuchar y hablar. Las dinámicas se organizaron a través del trabajo grupal para el desarrollo de los problemas. Bajo esta lógica los estudiantes deben ser capaces de resolver las diferencias de ideas que se producen en las etapas de análisis y diseño. Los estudiantes deben ser capaces de expresar (*hablar*) claramente sus puntos de vista en

cuanto a la comprensión del problema y sus ideas de diseño. Además, deben aprender a *escuchar* y ser respetuosos con las opiniones de sus compañeros, de tal manera de lograr acuerdos y alcanzar una propuesta de solución al problema planteado. Por otra parte, dada la propia naturaleza de la metodología, los estudiantes deben describir y documentar (*escribir*) las actividades que forman parte del diseño de la solución propuesta. En la figura-4, se observa parte de las actividades de la práctica.



Figura 4 – Comunicación.

Perspectiva. La validación de esta habilidad responde a la siguiente pregunta: “¿El trabajar en equipo me ha permitido desarrollar la habilidad de expresar mis ideas?”. Esta pregunta es evaluada con una escala de Likert (0-5). El resultado de la evaluación se muestra en figura-5.

El trabajar en equipo me dió un espacio para expresar mis ideas

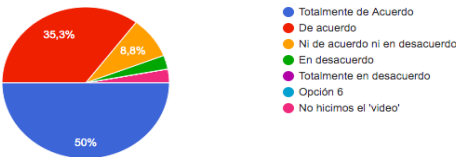


Figura 5 – Resultados.

Resolución de problemas:

Práctica. Uno de los principales focos de la metodología R&G, es desarrollar la capacidad de resolver problemas de programación a través de la aplicación de un proceso disciplinado y sistemático. Para lograr este objetivo, la metodología define cuatro pasos o actividades. Los estudiantes cada vez que deben resolver un problema de programación, deben seguir el procedimiento indicado por la metodología en cada una de sus etapas. En las dos primeras etapas, es en dónde se concentra el desarrollo de esta habilidad (tal como se aprecia en la tabla-1). En la primera etapa, se realiza un análisis

de la situación presentada para lograr determinar de manera precisa cuál es el problema que se debe resolver. En la segunda etapa, los estudiantes deben obtener, a través de refinamientos sucesivos, el conjunto de instrucciones que resuelve el problema. Para apoyar este proceso ambas etapas poseen canvas o plantillas para guiar el desarrollo. En la figura-6, se aprecia la práctica en los estudiantes.



Figura 6 – Resolución de problemas.

Perspectiva. De las consultas efectuadas a los estudiantes, estos han respondido que enfrentarse a los ejercicios es mucho más fácil con la metodología, ya que su estructura (etapas) y la secuencia ordenada de pasos, les facilita, primeramente, ordenar sus ideas y luego, desarrollarlas hasta obtener las instrucciones que forman parte de su programa. Algunas de las opiniones de los estudiantes son “Nos ayuda a ser mas estructurados con la metodología R y G”, “la metodología utilizada ha sido una gran ayuda para entender los contenidos”, “La obligación de tener que analizar claramente un problema para poder comenzar a resolverlo ha sido un aspecto positivo” o “aprender a pensar de manera estructurada para así poder comprender como funciona un sistema o programa.”.

5. Discusión

En el último tiempo se viene planteando la necesidad, de incorporar en los procesos formativos, el desarrollo de las denominadas habilidades blandas (Capretz & Ahmed, 2018). Hoy en día existen múltiples plataformas para aprender a programar, sin embargo éstas se centran en aspectos técnicos de la formación y no dan mayor cobertura al desarrollo de habilidades blandas en el estudiante. De la misma manera, los programas de enseñanza de las instituciones de educación se centran en la codificación y no el proceso del desarrollo de problemas, incluso basan sus cursos en las plataformas mencionadas anteriormente. Luego, la mayoría de los programas de enseñanza no están incluyendo actividades tendientes al desarrollo de las habilidades blandas.

Los resultados presentados en este estudio muestran que la metodología R&G se presenta como una herramienta para la enseñanza de la programación que es capaz de atender el desarrollo de las habilidades blandas en los estudiantes de programación, además de entregarles una guía para la resolución de problemas, la cual permite, por una parte, el

desarrollo de habilidades cognitivas para plantear soluciones, y, por otra parte, provee de las habilidades técnicas o disciplinares que todo desarrollador debe poseer.

Otro elemento favorable, es que la metodología introduce el desarrollo de estas habilidades desde etapas tempranas de la formación de los ingenieros, lo que ayuda al reforzamiento de las mismas durante todo el proceso de formación, lo que da más posibilidades de lograr un desarrollo eficaz de estas habilidades y permite asegurar que el estudiante llegue mejor preparado a la industria, tanto en lo técnico como en lo personal.

6. Limitaciones a la validez

Si bien la metodología ha sido utilizada y validada previamente, el análisis de la forma en que ésta aborda las habilidades blandas se realiza por primera vez en este semestre. Se realiza un análisis de la opinión que tienen los estudiantes en relación cómo la metodología les ayuda a comunicar sus propuestas (formato gráfico) y, sobre todo, por qué la resolución de problemas a través del trabajo en equipo se considera una mejor estrategia.

Además, cabe destacar que el universo de estudiantes observados, del cual reconocemos que no es suficiente para obtener conclusiones estadísticamente confiables, sin embargo dan una información cualitativa que permite establecer una tendencia y un futuro marco de investigación.

7. Conclusiones

Este artículo describe como la metodología utilizada para enseñar a programar, denominada metodología R&G (Rápida & Global), considera un conjunto de habilidades blandas, tales como resolución de problemas, trabajo en equipo y comunicación, en un primer curso de programación. Se considera un marco de análisis el cual considera 3 dimensiones: habilidad, práctica y perspectiva, para discutir la manera en que las habilidades de trabajo en equipo, comunicación y resolución de problemas son consideradas en la metodología. Los resultados presentados demuestran que es posible considerar metodologías que, desde un principio, aborden las habilidades blandas requeridas por los estudiantes de computación y solicitadas por la industria de software.

Trabajo futuro incluye realizar equivalencias entre las competencias requeridas por instituciones de acreditación, tales como ABET, así como un modelo conceptual que describa todos los aspectos necesarios a incluir dentro del proceso de enseñanza de programación, con el objeto de mejorar el entendimiento del proceso de enseñanza.

Agradecimientos

Trabajo financiado parcialmente por la Universidad de La Frontera, Proyecto IF18-0011.

Referencias

- ACM Curricula Recommendations: Computer Science Curricula 2013. (n.d.).
- Ahmed, F., Capretz, L. F., Bouktif, S., & Campbell, P. (2013). Soft Skills and Software Development: A Reflection from Software Industry. *International Journal of Information Processing and Management*, 4(3), 171–191. <https://doi.org/10.4156/ijipm.vol4.issue3.17>
- Ardis, M. A., Budgen, D., Hislop, G. W., Offutt, J., Sebern, M. J., & Visser, W. (2015). SE 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering. *IEEE Computer*, 48(11), 106–109.
- Belzer, K. (2004). Project management: Still more art than science. Retrieved August 19, 2004.
- Burke, Q., Bailey, C., Lyon, L. A., & Green, E. (2018). Understanding the Software Development Industry's Perspective on Coding Boot Camps Versus Traditional 4-year Colleges. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education* (pp. 503–508). New York, NY, USA: ACM. <https://doi.org/10.1145/3159450.3159485>
- Capretz, L. F., & Ahmed, F. (2018). A Call to Promote Soft Skills in Software Engineering. *ArXiv Preprint ArXiv:1901.01819*.
- Chilana, P. K., Alcock, C., Dembla, S., Ho, A., Hurst, A., Armstrong, B., & Guo, P. J. (2015). Perceptions of non-CS majors in intro programming: The rise of the conversational programmer. In *2015 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)* (pp. 251–259). <https://doi.org/10.1109/VLHCC.2015.7357224>
- Denning, P. J., & Gordon, E. E. (2015). A Technician Shortage. *Commun. ACM*, 58(3), 28–30. <https://doi.org/10.1145/2723673>
- Diéguez, M., Gacitúa, R., & Vidal, E. (2018). Evaluating the impact of training software engineers using the R&G methodology. In *JIIISIC 2018 - Jornadas Iberoamericanas de Ingeniería de Software e Ingeniería del Conocimiento* (pp. 185–196). Retrieved from <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85050094817&partnerID=40&md5=ac82dbf7adfe3a438a408b3036104584>
- Gacitúa, R., Diéguez, M., & Vidal, E. (2017). Forming software architects in early stages: From craft to engineering. In *2017 36th International Conference of the Chilean Computer Science Society (SCCC)* (pp. 1–8). <https://doi.org/10.1109/SCCC.2017.8405130>
- Hazzan, O., & Har-Shai, G. (2013). Teaching Computer Science Soft Skills As Soft Concepts. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education* (pp. 59–64). New York, NY, USA: ACM. <https://doi.org/10.1145/2445196.2445219>

- Hohpe, G., Ozkaya, I., Zdun, U., & Zimmermann, O. (2016). The software architect's role in the digital age. *IEEE Software*, 33(6), 30–39.
- Kamin, M. (2013). *Soft skills revolution: a guide for connecting with compassion for trainers, teams, and leaders*. John Wiley & Sons.
- Kirschner, P., Sweller, J., & Clark, R. (2006). Why Minimal Guidance During Instruction Does Not Work: An Analysis of the Failure of Constructivist, Discovery, Problem-Based, Experiential, and Inquiry-Based Teaching. *Educational Psychologist*, 41. https://doi.org/10.1207/s15326985ep4102_1
- Lovett, M. C. (2002). Problem solving. *Stevens' Handbook of Experimental Psychology*.
- Makasiranondh, W., Maj, S., & D, V. (2011). Student Opinions on their Development of Non-technical Skills in IT Education. *Modern Applied Science*, 5. <https://doi.org/10.5539/mas.v5n2p3>
- Mayer, R. E., & Wittrock, M. C. (2006). Problem solving. *Handbook of Educational Psychology*, 2, 287–303.
- Milne, I., & Rowe, G. (2002). Difficulties in learning and teaching programming—views of students and tutors. *Education and Information Technologies*, 7(1), 55–66.
- Parnas, D. L. (2010). Risks of Undisciplined Development. *Commun. ACM*, 53(10), 25–27. <https://doi.org/10.1145/1831407.1831419>
- Robles, M. M. (2012). Executive perceptions of the top 10 soft skills needed in today's workplace. *Business Communication Quarterly*, 75(4), 453–465.
- Tomić, B., Jovanović, J., Milikić, N., Devedžić, V., Dimitrijević, S., Đurić, D., & Ševarac, Z. (2019). Grading students' programming and soft skills with open badges: A case study. *British Journal of Educational Technology*, 50(2), 518–530. <https://doi.org/10.1111/bjet.12564>
- Vidal, E., Gacitúa, R., Diéguez, M., & Cachero, C. (2019). Correspondence analysis between programming teaching approaches. In *XIV Jornadas Iberoamericanas de Ingeniería de Software e Ingeniería del Conocimiento, IIISIC 2019* (pp. 177–188).

© 2020. This work is published under
<https://creativecommons.org/licenses/by-nc-nd/4.0/>(the
“License”). Notwithstanding the ProQuest Terms and
Conditions, you may use this content in accordance with the
terms of the License.